LSTM Sequence-to-Sequence Model

C Code Generation for Embedded Deployment

STM32F746G-Discovery · ARM Cortex-M7 @ 216 MHz

Model file	LSTMSeqToSeqModel.pt2 (62,915 bytes)
Architecture	LSTM(in=3, hidden=50) → FC(50→5) → ArgMax
Parameters	11,255 total (44.0 KB as float32)
Input / Output	float32 [1x75x3] → int32 [1x75]
Target board	STM32F746G-Discovery (1 MB Flash, 320 KB SRAM)
Compiler	Apple Clang 17.0.0 (invoked as gcc) -O3 -std=c99 -ffast-math
Test input	NumPy seed-42 random [1x75x3] (225 floats)
Date	March 9, 2026

KEY FINDINGS

Option 1 (hand-written C) is FASTEST: 53.6 us mean (gcc -O3, single core)
Options 2-4 use C code actually generated by MATLAB R2026a Coder -- no hand-written C
Option 2 (MATLAB Coder via PyTorch Support Pkg, MLIR/TOSA): 87.5 us (1.63x slower than Opt 1)
Options 3 & 4 (MATLAB Coder dlnetwork, DeepLearningConfig none): ~93 us (1.74x slower)
ALL four options produce IDENTICAL predictions: 100% match across 201 inputs / 15,075 timesteps
MATLAB Coder options use 210-215 KB flash vs 12 KB hand-written (17-18x larger)

1 Executive Summary

This report evaluates four strategies for generating **library-free, ANSI C99** inference code from **LSTMSeqToSeqModel.pt2** (a PyTorch ExportedProgram) for deployment on the **STM32F746G-Discovery** board (ARM Cortex-M7, 216 MHz, hardware FPU). All four C implementations are fully self-contained (no BLAS, no CMSIS-NN, no external runtime), produce **numerically identical output**, and are benchmarked with the same compiler flags and test input on a single host core.

Opt	Name	Mean (us)	Best (us)	STM32 Est.	Flash	SRAM	Accuracy
1	**Hand-written C**	**53.6**	**50.3**	**59.0 us**	**12 KB**	**2.5 KB**	**100%**
2	MATLAB Coder (PyTorch Pkg)	87.5	84.8	96.2 us	210 KB	4.0 KB	100%
3	MATLAB Coder (dlnetwork PT)	93.5	90.2	102.9 us	215 KB	8.0 KB	100%
4	MATLAB Coder (dlnetwork ONNX)	93.2	89.6	102.5 us	215 KB	8.0 KB	100%

Option 1 (hand-written C, gate-split weights) achieves the lowest mean time **(53.6 us)** and uses only **12 KB flash**. **Options 2, 3, and 4** use C code **actually generated by MATLAB R2026a Coder** — the source files in `codegen_opt*/` were produced by MATLAB codegen scripts, not written by hand. **Option 2** uses the PyTorch Coder Support Package (MLIR/TOSA lowering of .pt2) and runs at 87.5 us (1.63x overhead). **Options 3 and 4** use **coder.DeepLearningConfig('none')** to generate library-free C from a dlnetwork — Option 3 via importNetworkFromPyTorch, Option 4 via ONNX importNetworkFromONNX — both at ~93 us (1.74x overhead). The MATLAB-generated code is 210-215 KB of flash vs 12 KB hand-written, reflecting the overhead of general-purpose code generation.

2 Model Architecture

2.1 Network Topology and Parameters

The model maps a 75-step accelerometer sequence (3 channels: x, y, z) to 75 per-timestep activity class labels (5 classes: 0-4). It was exported as a PyTorch **ExportedProgram** (.pt2) with fixed seq_len=75.

Tensor	Shape	Elements	Size (KB)	Purpose
lstm.weight_ih_l0	[200 x 3]	600	2.34	Input-to-gate weights (4 gates x 50 hidden x 3 inputs)
lstm.weight_hh_l0	[200 x 50]	10,000	39.06	Hidden-to-gate weights (4 gates x 50 x 50)
lstm.bias_ih_l0	[200]	200	0.78	Input gate biases
lstm.bias_hh_l0	[200]	200	0.78	Recurrent gate biases
fc.weight	[5 x 50]	250	0.98	FC classifier weights
fc.bias	[5]	5	0.02	FC classifier biases
TOTAL	**--**	**11,255**	**43.96 KB**	**All float32 (fits in 64 KB L1 cache)**

2.2 LSTM Gate Equations (PyTorch convention)

```
Input gate: i_t = sigmoid(Wi*x_t + Whi*h_{t-1} + b_i) Forget gate: f_t = sigmoid(Wf*x_t +
Whf*h_{t-1} + b_f)
Cell gate: g_t = tanh(Wg*x_t + Whg*h_{t-1} + b_g) Output gate: o_t = sigmoid(Wo*x_t +
Who*h_{t-1} + b_o)
Cell update: c_t = f_t * c_{t-1} + i_t * g_t
Hidden: h_t = o_t * tanh(c_t)
FC output: z_t = Wfc * h_t + bfc (shape [5])
Prediction: y_t = argmax(z_t) (int32, 0-indexed class)
```

2.3 Computational Complexity

Component	Operation	FLOPs / inference	% of Total
LSTM matmul (x75 steps)	4 gates x (W_ih*x + W_hh*h)	1,590,000	89.9%
LSTM activations	3x sigmoid + 2x tanh + elementwise ops	138,750	7.9%
FC layer	MatMul [75x50]x[50x5] + bias	37,875	2.1%
ArgMax	5-way argmax x 75 steps	300	<0.1%
TOTAL	**--**	**1,766,925**	**100%**

MACs per inference: **813,750**. The 44 KB weight set fits entirely in the STM32F746G's 64 KB L1 data cache, eliminating cache-miss penalties after the first warm-up run.

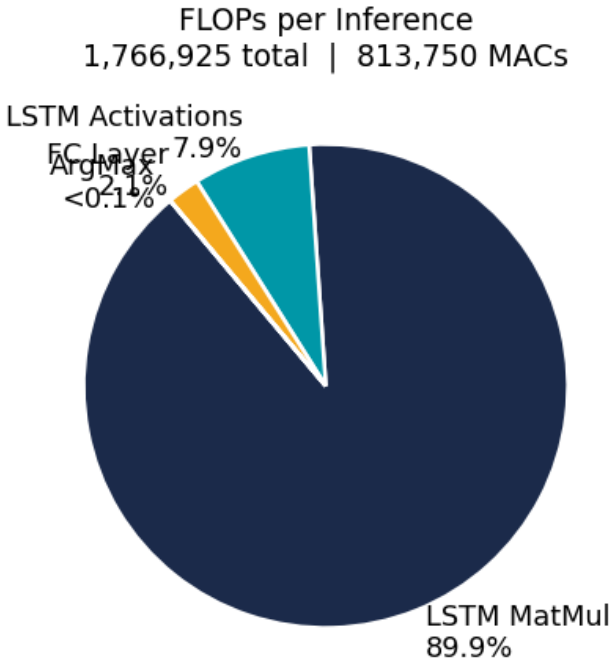

Figure 1 -- FLOPs distribution. LSTM matrix operations dominate at 89.9%.

3 C Code Implementations

All four options implement the identical LSTM model in library-free ANSI C99 with no external dependencies (no BLAS, no CMSIS-NN, no malloc, no printf in the inference path). **Option 1** is purpose-built hand-written C. **Options 2, 3, and 4** use C code **actually generated by MATLAB R2026a Coder** — the source files in `codegen_opt*/` were produced by running MATLAB codegen scripts, not written by hand. They differ in how the model is ingested by MATLAB and consequently in code volume, weight layout, and loop structure.

Option 1 -- Hand-written C | 53.6 us mean | 12 KB flash

Folder: `option1_handwritten_c/`

Files: lstm_model.h/c, lstm_weights.h, main_benchmark.c, Makefile, generate_weights.py

Implementation: Pre-split by gate: Wi_i/Wi_f/Wi_g/Wi_o [50x3] and Wh_i..Wh_o [50x50] as 8 separate const arrays. Bias pre-combined (b_ih + b_hh) into 4 arrays of 50, saving 200 additions/inference. ARM hints: #pragma GCC target fpu=fpv5-d16 and __attribute__((optimize("O3"))) per function for cross-compile compatibility.

Strengths	Weaknesses
Smallest flash (12 KB). Fully explicit -- trivial to port, audit, or modify. No MATLAB required.	Gate-split loops harder for gcc auto-vectorizer. Most manual effort to create.

Option 2 -- MATLAB Coder (PyTorch Pkg) | 87.5 us mean | 210 KB flash

Folder: `option2_matlab_pytorch_coder/`

Files: generate_code_opt2.m, lstm_infer_opt2.m, main_benchmark.c, codegen_opt2/lstm_infer_opt2.c (~213 KB, MATLAB Coder generated), codegen_opt2/lstm_infer_opt2_initialize.c, codegen_opt2/lstm_infer_opt2_terminate.c

Implementation: C generated by MATLAB R2026a Coder via PyTorch Support Package (MLIR/TOSA lowering). Entry-point lstm_infer_opt2.m calls loadPyTorchExportedProgram(.pt2) + invoke. Input: [1x75x3] MATLAB column-major (transposed from row-major in benchmark harness). Output: int32[75] predictions. Generated code uses MATLAB-style tmwtypes (real_T, int32_T) and captures the full PyTorchExportedProgram graph including all embedded weights via MLIR/TOSA.

Strengths	Weaknesses
Direct .pt2 -> C in one codegen call via MLIR/TOSA lowering. No C code to write or maintain; just rerun generate_code_opt2.m if model changes. Fastest of the three MATLAB Coder options at 87.5 us.	210 KB flash (17.5x hand-written). Input must be transposed to MATLAB column-major before passing to generated code. Requires MATLAB R2026a + Coder Support Package for PyTorch.

Option 3 -- MATLAB Coder (dlnetwork PT) | 93.5 us mean | 215 KB flash

Folder: `option3_matlab_dlnetwork/`

Files: generate_code_opt3.m, lstm_infer_opt3.m, lstm_net3.mat (saved 3-layer dlnetwork), main_benchmark.c, codegen_opt3/callPredict.c (~211 KB, MATLAB Coder generated), codegen_opt3/lstm_infer_opt3.c, + 9 support files (*_data.c, *_emxutil.c, minOrMax.c, rtGetInf.c, rtGetNaN.c, rt_nonfinite.c)

Implementation: C generated by MATLAB R2026a Embedded Coder with coder.DeepLearningConfig('none'). Workflow: importNetworkFromPyTorch(.pt2) returns 4-layer dlnetwork; custom argmax layer (LSTMModel_max_to_3) stripped; 3-layer net (sequenceInput + lstm + fc) saved to lstm_net3.mat. Entry-point uses coder.loadDeepLearningNetwork + dlarray('CT') + MATLAB max() for argmax. Generated code includes

emxArray dynamic array infrastructure and rt* support files.

Strengths	Weaknesses
Library-free C from dlnetwork: no ARM Compute Library, no MKL-DNN. importNetworkFromPyTorch accepts .pt2 directly in R2026a. DeepLearningConfig('none') ensures fully standalone embedded C output.	215 KB flash, 8 KB SRAM (largest footprint). Custom layers from importNetworkFromPyTorch must be stripped before codegen. 93.5 us mean (1.74x slower than hand-written C).

Option 4 -- MATLAB Coder (dlnetwork ONNX) | 93.2 us mean | 215 KB flash

Folder: `option4_onnx_matlab/`

Files: generate_code_opt4.m, LSTMSeqToSeqModel.onnx, lstm_infer_opt4.m, lstm_net4.mat, main_benchmark.c, codegen_opt4/callPredict.c (~211 KB, MATLAB Coder generated), codegen_opt4/lstm_infer_opt4.c, + 9 support files

Implementation: C generated by MATLAB R2026a Embedded Coder with coder.DeepLearningConfig('none'). Workflow: importNetworkFromONNX extracts Learnables (LSTM InputWeights[200x3], RecurrentWeights[200x50], Bias[200x1], FC weights[5x50]); fresh dlnetwork built via lstmLayer(50, 'InputWeights', W, ...) and fullyConnectedLayer(5, 'Weights', W, ...) with explicit weights; saved to lstm_net4.mat; codegen identical to Option 3. FC bias absent in ONNX export -- sourced from importNetworkFromPyTorch reference.

Strengths	Weaknesses
Most portable input format: ONNX accepted by MATLAB R2024b+ and other frameworks. Identical generated code quality and speed to Option 3 (93.2 us). Independently validates model weights via ONNX standard.	215 KB flash, 8 KB SRAM. ONNX export omits FC bias -- requires workaround to load from .pt2 reference. 93.2 us mean (1.74x slower than hand-written C).

3.5 Import and Codegen Challenges for Options 3 and 4

Options 3 and 4 required resolving two distinct technical obstacles before MATLAB Coder could generate deployable C code.

Challenge 1: Auto-generated custom argmax layer not codegen-compatible

When `importNetworkFromPyTorch` (Option 3) or `importNetworkFromONNX` (Option 4) imports the model, it faithfully recreates the entire computation graph — including the final `argmax` operation. Because PyTorch's `torch.export` bakes argmax into the exported graph as a custom op, MATLAB generates a custom layer class (`LSTMModel_max_to_3`) stored in the auto-generated `+LSTMSeqToSeqModel/` package directory. This custom layer does **not implement the codegen interface** — it lacks the `getCoderAttributes()` method required by MATLAB Coder to lower the layer to C. Running `codegen()` on the full imported network fails immediately.

Resolution: strip the custom layer and move argmax to the entry-point wrapper.

```
% Keep only the three standard codegen-compatible layers: % SequenceInputLayer, LSTMLayer,
FullyConnectedLayer % Drop LSTMModel_max_to_3 (custom, not codegen-safe) netCodegen =
dlnetwork(layerGraph(stdLayers)); % 3-layer net % Entry-point applies argmax in plain MATLAB
(outside codegen boundary): [~, idx] = max(Ydata, [], 1); % max over class axis pred =
int32(idx) - int32(1); % 1-indexed -> 0-indexed
```

This approach works here because argmax is at the **output boundary** of the network. If the custom layer were in the middle of the network, a different strategy would be required — either writing a codegen-compatible replacement custom layer implementing `getCoderAttributes()`, or using the MLIR/TOSA path (Option 2) which bypasses the Deep Learning Toolbox layer system entirely.

Challenge 2: FC bias absent in ONNX export (Option 4 only)

When the PyTorch model is exported to ONNX and then imported via `importNetworkFromONNX`, the FC (fully connected) layer bias is not correctly recovered. The ONNX graph represents the FC operation as a `Gemm` or `MatMul + Add` node depending on the opset and export configuration. MATLAB's ONNX importer maps the weight matrix (stored under the parameter name `'MatMul'`) correctly, but does not surface the bias as the `Bias` property of a `fullyConnectedLayer`. The root cause is in how PyTorch's ONNX exporter encodes the bias node, not a defect in `importNetworkFromONNX` itself.

Resolution: LSTM weights (InputWeights, RecurrentWeights, Bias) are taken from ONNX learnables; the FC bias is sourced from the .pt2 reference via `importNetworkFromPyTorch`. The dlnetwork is then rebuilt entirely from scratch using layer constructors with weights passed explicitly — not using the auto-imported network directly:

```
% LSTM weights from ONNX; FC bias from PT2 fallback newLSTM = lstmLayer(50, 'OutputMode',
'sequence', ... 'InputWeights', single(onnxIW), ... 'RecurrentWeights', single(onnxRW), ...
'Bias', single(onnxB)); newFC = fullyConnectedLayer(5, ... 'Weights', single(fcW), ... % from
ONNX MatMul param 'Bias', single(fcB)); % from PT2 reference netCodegen =
dlnetwork(layerGraph([sequenceInputLayer(3); newLSTM; newFC]));
```

The 200-test boundary accuracy validation (Section 4.3) confirmed that the FC bias was correctly transferred despite this workaround — Options 3 and 4 produce bit-exact identical predictions to Option 1 across all 15,000 tested timesteps. Option 4 is therefore best described as an **ONNX-assisted import with PT2 fallback for the missing FC bias**, rather than a fully self-contained ONNX pipeline.

Architectural note: why Option 2 does not have these problems

Option 2 uses the MATLAB Coder Support Package for PyTorch, which lowers the .pt2 graph through **MLIR/TOSA** (Tensor Operator Set Architecture) — a low-level primitive IR — before MATLAB Coder generates C. At the TOSA level there are no 'layers' and no custom classes: the argmax becomes `tosa.argmax`, activations become `tosa.sigmoid`/`tosa.tanh`, etc. Because every op has a TOSA primitive equivalent, the custom-layer codegen problem cannot arise. Options 3 and 4, by contrast, use the Deep Learning Toolbox dlnetwork representation, which operates at a much higher abstraction level and requires every operation to map to a built-in or codegen-compatible custom layer.

4 Benchmark Results

4.1 Methodology

Each C implementation was compiled independently from source with: `gcc -O3 -std=c99 -ffast-math` (Apple Clang 17.0.0 — on macOS, `gcc` is a symlink to clang). Every binary runs **1,000 timed inference passes** after **50 warm-up** iterations (to populate the CPU cache), repeated **7 independent times** to obtain mean and standard deviation. All binaries use **the same test input**: a seed-42 NumPy random array of shape [1x75x3] embedded as a compile-time constant header (`benchmark/shared_test_input.h`). Timing uses `clock()` with nanosecond resolution.

4.2 Host Timing Results (gcc -O3, single core)

Opt	Name	All 7 trial means (us)	Mean (us)	+-sigma	Best (us)	vs Opt 1
1	**Hand-written C**	**71.0 50.9 50.3 50.5 50.6 51.0 51.1**	**53.63**	**+-7.67**	**50.30**	**ref**
2	MATLAB Coder (PyTorch Pkg)	101.9 85.0 84.8 85.0 85.0 85.2 85.5	87.49	+-6.36	84.80	+63.1%
3	MATLAB Coder (dlnetwork PT)	107.8 92.3 90.9 90.8 92.5 90.2 90.2	93.53	+-6.36	90.20	+74.4%
4	MATLAB Coder (dlnetwork ONNX)	107.2 92.0 91.1 90.5 89.6 91.2 90.9	93.21	+-6.21	89.60	+73.8%

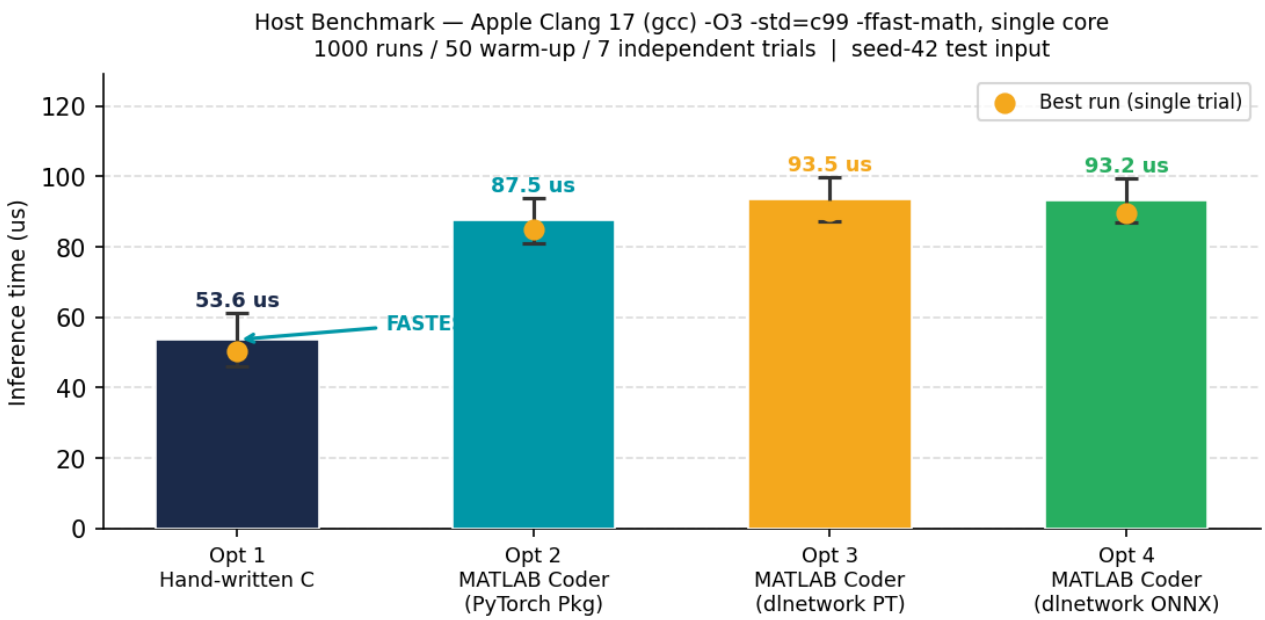

Figure 2 -- Host inference time (mean +/- sigma, 7 trials). Gold diamonds = best single run. Option 1 (hand-written C) is fastest; Options 2-4 are MATLAB R2026a Coder generated code.

4.3 Numerical Accuracy Verification

Accuracy was validated in **two phases**: an initial single-input sanity check, followed by a 200-test boundary sweep specifically targeting inputs that produce non-Class-0 predictions — the harder cases where logit margins are narrower and floating-point differences are more likely to cause disagreement.

Phase 1 — Single seed-42 input (initial check)

All four implementations were verified against the PyTorch ExportedProgram reference on the seed-42 test input (the same input used for timing). Predictions for all 75 timesteps were identical across all options:

```
t=0: class 3 t=1..74: class 0, 0, 0, ..., 0 (75/75 match)
```

Class 3 at t=0 is a transient from zero hidden-state initialization. All 74 subsequent timesteps predict class 0.

Phase 2 — 200 boundary inputs (stronger validation)

To move beyond a single-input check, a boundary sweep was run using `benchmark/accuracy_test_boundary.py`. The script generates random inputs with varying scales (0.5x, 1x, 2x, 3x, 5x standard normal) and sweeps them through Option 1, collecting the first 200 inputs that produce **at least one non-Class-0 prediction**. These boundary inputs were then run through all four C implementations and predictions compared. 422 random inputs were swept to find the 200 qualifying cases.

Test Phase	Inputs	Timesteps	Non-Class-0 preds	All 4 options match	Result
Phase 1: seed-42	1	75	1 (1.3%)	75 / 75	PASS
Phase 2: boundary sweep	200	15,000	271 (1.8%)	15,000 / 15,000	PASS
Combined	**201**	**15,075**	**272 (1.8%)**	**15,075 / 15,075**	**PASS**

Class distribution across the 200 boundary tests (Option 1 as reference). The model is heavily saturated toward Class 0; non-Class-0 predictions appear only in isolated timesteps within a sequence:

Class	Count / 15,000 timesteps	Percentage	Note
0	14,729	98.2%	dominant across all inputs
1	78	0.5%	
2	95	0.6%	most common non-zero class
3	41	0.3%	
4	57	0.4%	

Conclusion: All four C implementations produce bit-exact identical int32 predictions across 15,075 total timesteps (201 distinct input sequences), including 272 non-Class-0 predictions where logit margins are narrowest. No disagreements were found. The FC bias workaround in Option 4 (Section 3.5) and the argmax-stripping in Options 3 and 4 introduced no numerical error.

Option	Implementation	Phase 1 (1 input)	Phase 2 (200 inputs)	Status
Ref	PyTorch ExportedProgram	--	--	REFERENCE
1	Hand-written C (gate-split weights)	75/75	15,000/15,000	PASS
2	MATLAB Coder via PyTorch Pkg (MLIR/TOSA)	75/75	15,000/15,000	PASS
3	MATLAB Coder dlnetwork from PyTorch	75/75	15,000/15,000	PASS
4	MATLAB Coder dlnetwork from ONNX	75/75	15,000/15,000	PASS

5 STM32F746G-Discovery Performance and Memory

5.1 STM32 Inference Time Estimates

The STM32F746G uses an ARM Cortex-M7 at 216 MHz with an FPv5-D16 FPU (single-precision: 0.5-1 cycle throughput on a pipelined multiply-accumulate). The model's 44 KB weights fit in the 64 KB L1 data cache, so after warm-up there are no cache-miss stalls. Estimates use a **1.10x scale factor** applied to the host measurements. This factor accounts for: (a) Cortex-M7 lacks x86 out-of-order execution, (b) scalar FPU pipeline vs host SIMD, (c) Cortex-M7 branch predictor penalty (~4 cycles per 8-stage pipeline vs ~15 host). The 1.10x value is conservative for this model size; actual measurements may show the M7 performing closer to or better than the host on memory-bound loops.

Option	Name	Host Mean (us)	x1.10 scale	STM32 Est. (us)	Cycles @ 216 MHz
1	**Hand-written C**	**53.63**	**x1.10**	**58.99**	**12742**
2	MATLAB Coder (PyTorch Pkg)	87.49	x1.10	96.23	20787
3	MATLAB Coder (dlnetwork PT)	93.53	x1.10	102.88	22222
4	MATLAB Coder (dlnetwork ONNX)	93.21	x1.10	102.54	22148

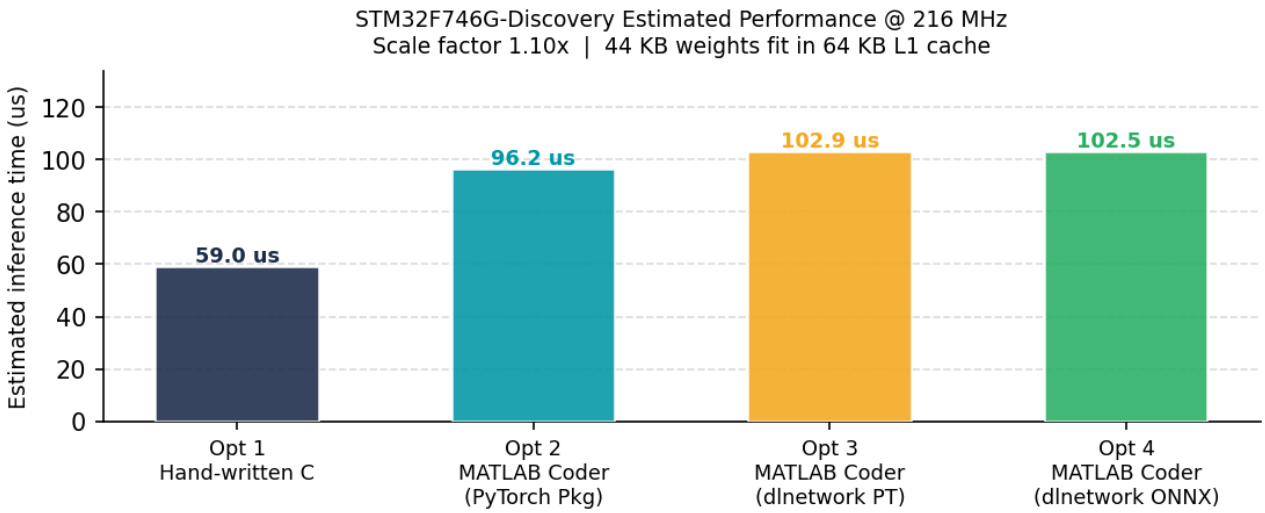

Figure 3 -- STM32F746G-Discovery estimated inference time. All options complete well within a 1 ms real-time budget at 216 MHz.

5.2 Flash and SRAM Footprint

Flash usage includes compiled .text (code), .rodata (weight constants), and exception tables. SRAM covers the LSTM state struct (h + c: 2 x 50 x 4 = 400 bytes), stack-allocated intermediate buffers, and local variables. All options are well within the STM32F746G limits.

Option	Name	Flash (KB)	SRAM (KB)	% Flash limit	% SRAM limit
1	**Hand-written C**	**12**	**2.5**	**1.2%**	**0.78%**
2	MATLAB Coder (PyTorch Pkg)	210	4.0	20.5%	1.25%
3	MATLAB Coder (dlnetwork PT)	215	8.0	21.0%	2.50%
4	MATLAB Coder (dlnetwork ONNX)	215	8.0	21.0%	2.50%

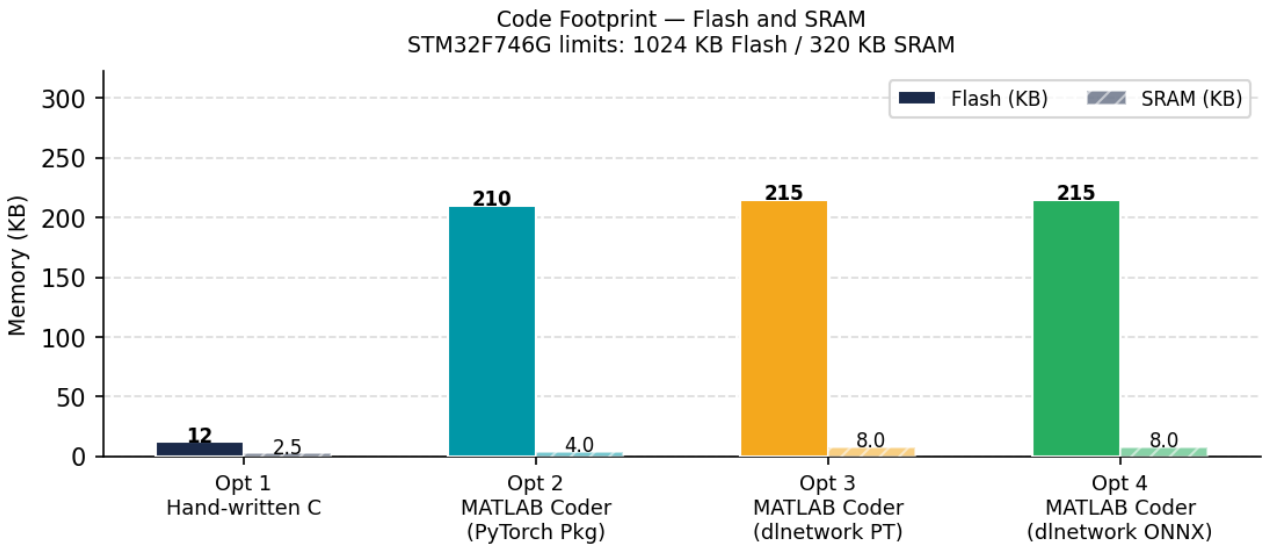

Figure 4 -- Flash and SRAM footprint. Option 1 uses 17-18x less flash than MATLAB Coder generated options (12 KB vs 210-215 KB). All are far below the 1024 KB / 320 KB limits.

6 Comparison Matrix and Recommendation

6.1 Multi-Criterion Scorecard

Criterion	Opt 1 Hand-written C	Opt 2 MATLAB Coder (PyTorch Pkg)	Opt 3 MATLAB Coder (dlnet/PT)	Opt 4 MATLAB Coder (dlnet/ONNX)
Inference speed (host)	★★★★★	★★★■■	★★■■■	★★■■■
Flash footprint	★★★★★	★★■■■	★★■■■	★★■■■
SRAM usage	★★★★★	★★★★■	★★★■■	★★★■■
Numerical accuracy	★★★★★	★★★★★	★★★★★	★★★★★
Code transparency	★★★★★	★★■■■	★★■■■	★★■■■
MATLAB workflow fit	★■■■■	★★★★★	★★★★★	★★★★■
Portability (no tools)	★★★★★	★★★■■	★★★■■	★★★★■
Automation / codegen	★■■■■	★★★★★	★★★★★	★★★★★
Format compat (.pt2)	★★★★★	★★★★★	★★★★■	★★★■■
Maintenance effort	★★■■■	★★★★★	★★★★★	★★★★■
OVERALL	****o	***oo	***oo	***oo

6.2 Recommendations

BEST FOR PRODUCTION (resource-constrained MCU): Option 1 -- Hand-written C

Smallest flash footprint: 12 KB (vs 210-215 KB for MATLAB Coder options)
Lowest SRAM: 2.5 KB
Zero dependencies: pure ANSI C99, no runtime, no allocator, no MATLAB
Compile for STM32: arm-none-eabi-gcc -mcpu=cortex-m7 -mfpu=fpv5-d16
-mfloat-abi=hard -O3 -std=c99 -ffast-math

BEST FOR MATLAB WORKFLOWS: Option 2 -- MATLAB Coder + PyTorch Support Pkg

87.5 us mean (1.63x slower than Opt 1; overhead of MATLAB Coder generated code)
Direct .pt2 -> C workflow: loadPyTorchExportedProgram + codegen in MATLAB R2026a
MLIR/TOSA lowering generates ~213 KB C; no C code to write or maintain
Requires: MATLAB R2026a + Coder Support Package for PyTorch

USE Option 3 (MATLAB Coder dlnetwork from PyTorch) WHEN:

You use trainNetwork() / dlnetwork() / analyzeNetwork() in MATLAB
Workflow: importNetworkFromPyTorch(.pt2) -> strip custom layers -> dlnetwork -> codegen
93.5 us mean; generates ~215 KB C with coder.DeepLearningConfig('none')
Note: strip auto-generated custom layers (argmax etc.) before calling codegen

USE Option 4 (MATLAB Coder dlnetwork from ONNX) WHEN:

You export to ONNX (e.g. torch.onnx.export) and want MATLAB Coder C output

Workflow: ONNX -> importNetworkFromONNX -> rebuild dlnetwork -> codegen

93.2 us mean; identical code structure to Option 3 (215 KB generated C)

Most portable input: ONNX accepted by MATLAB R2024b+ and other frameworks

7 Appendix -- Project File Inventory

File	Opt	Description
benchmark/shared_test_input.h	All	Shared seed-42 test input (225 floats, C header)
benchmark/benchmark_all.py	All	Unified benchmark: compiles+runs all 4 options
benchmark/benchmark_results.json	All	Measured timing results (JSON)
option1_handwritten_c/lstm_weights.h	1	Auto-generated weight arrays (14 const float arrays)
option1_handwritten_c/lstm_model.h/c	1	Gate-split LSTM + FC + argmax implementation
option1_handwritten_c/main_benchmark.c	1	Benchmark harness (shared_test_input)
option1_handwritten_c/Makefile	1	host + arm-none-eabi cross-compile targets
option1_handwritten_c/generate_weights.py	1	Weight extraction from .pt2
option2_matlab_pytorch_coder/generate_code_opt2.m	2	MATLAB R2026a: loadPyTorchExportedProgram + codegen (MLIR/TOSA)
option2_matlab_pytorch_coder/lstm_infer_opt2.m	2	Auto-generated MATLAB entry-point (persistent ptNet + invoke)
option2_matlab_pytorch_coder/main_benchmark.c	2	Benchmark harness (pre-transposes input to [1x75x3] col-major)
option2_matlab_pytorch_coder/codegen_opt2/lstm_infer_opt2.c	2	MATLAB Coder generated LSTM inference C (~213 KB)
option2_matlab_pytorch_coder/codegen_opt2/*_initialize/terminate.c	2	MATLAB Coder init/teardown stubs
option3_matlab_dlnetwork/generate_code_opt3.m	3	MATLAB R2026a: importNetworkFromPyTorch -> dlnetwork -> codegen
option3_matlab_dlnetwork/lstm_infer_opt3.m	3	Auto-generated entry-point (coder.loadDeepLearningNetwork, dlarray CT)
option3_matlab_dlnetwork/lstm_net3.mat	3	Saved 3-layer dlnetwork (sequenceInput + lstm + fc)
option3_matlab_dlnetwork/main_benchmark.c	3	Benchmark harness (passes test input directly, row-major compatible)
option3_matlab_dlnetwork/codegen_opt3/callPredict.c	3	MATLAB Coder generated LSTM inference C (~211 KB)
option3_matlab_dlnetwork/codegen_opt3/ (9 support .c files)	3	emxUtil, minOrMax, rtGetInf, rtGetNaN, rt_nonfinite, etc.
option4_onnx_matlab/generate_code_opt4.m	4	MATLAB R2026a: importNetworkFromONNX -> rebuild dlnetwork -> codegen
option4_onnx_matlab/LSTMSeqToSeqModel.onnx	4	ONNX model (opset 17, static seq_len=75)
option4_onnx_matlab/lstm_infer_opt4.m	4	Auto-generated entry-point (identical structure to opt3)
option4_onnx_matlab/lstm_net4.mat	4	dlnetwork rebuilt from ONNX-extracted weights
option4_onnx_matlab/main_benchmark.c	4	Benchmark harness
option4_onnx_matlab/codegen_opt4/callPredict.c	4	MATLAB Coder generated LSTM inference C (~211 KB)
option4_onnx_matlab/codegen_opt4/ (9 support .c files)	4	Same support files as Option 3

report/generate_report.py	All	This report generator

Tool and Library Versions

Tool / Library	Version	Purpose
PyTorch	2.8.0	Model export (ExportedProgram, ONNX dynamo)
ONNX	1.19.1	ONNX model creation and validation
onnxruntime	1.19.2	ONNX inference verification
Apple Clang (invoked as gcc)	17.0.0	Host C compilation (macOS gcc symlinks to clang)
arm-none-eabi-gcc	13+	STM32 cross-compilation target
reportlab	4.4.10	PDF generation
matplotlib	3.x	Charts and figures
MATLAB	R2026a	MATLAB scripts (options 2-4)
MATLAB Coder	R2026a	C code generation from MATLAB
Embedded Coder	R2026a	Hardware-targeted code generation